A Exemplar Queries

```
-- Extract keywords from a content field
SELECT id, s'Extract the main keywords from {content}' AS
↪  extracted_keywords
FROM documents;
-- Create a semantic summary from multiple fields
SELECT s'A summary of the event based on {title} and
↪  {description}'
AS event_summary FROM news_articles;
```

Figure 7: Example of Semantic Projection

```
-- Filter rows based on the sentiment of a specific field
SELECT * FROM product_reviews
WHERE s'The {review_text} expresses positive sentiment';
```

Figure 8: Example of Semantic Filter

```
-- Analyze the language style of the conference paper title
WITH title_style AS (
    SELECT Title, s'Analyze the language style features of paper
    ↪  title {Title}.
    Describe the key characteristics such as: formality level,
    technical terminology usage, length, structure, and word
    ↪  choice.'
    AS style_features
    FROM Paper WHERE ConferenceId != 0 AND Title IS NOT NULL
)
SELECT sem_agg(s'Based on these conference paper titles {Title}
and their style analyses {style_features}, summarize the overall
language style and terminology characteristics of conference
↪  paper titles.')
FROM title_style;
```

Figure 9: Example of Semantic Aggregate

```
-- Join two tables based on semantic similarity of their text
↪  fields
SELECT * FROM news_articles a JOIN blog_posts b
ON s'the topic of {a.headline} is related to {b.title}';
-- A complex hybrid join with both syntactic and semantic
↪  conditions
SELECT e.employee_id, eh.manager_id
FROM employees e INNER JOIN employee_hierarchy eh
ON e.manager_id = eh.employee_id
AND s'the {e.title} is a technical role and the hierarchy level
↪  {eh.level} < 3';
```

Figure 10: Example of Semantic Join

B Algorithms

Algorithm 2: Natural Language Expression Preprocessing and Reduction Process

Input : userInputNLE, dbSchema
Output : finalSQLFragments or processedNLE

// Step 1: Preprocessing with Database Context
intermediateNLE ← CleanRedundancies(userInputNLE)
cleanedNLE ← CorrectFields(intermediateNLE, dbSchema)
// Step 2: Reduction Assessment
prelimStatus ← AssessReduction(cleanedNLE)
if *prelimStatus = "not_reducible"* **then**
  return processedNLE ← cleanedNLE
reductionStatus ← AssessDbReduction(cleanedNLE, dbSchema)
if *reductionStatus = "not_reducible"* **then**
  return processedNLE ← cleanedNLE
candidateSQL ← GenerateSQL(cleanedNLE, dbSchema)
expandedSQL ← ExpandValues(candidateSQL, dbSchema)
return finalSQLFragments ← expandedSQL

Algorithm 3: Expression Exploration and Pairwise Similarity Estimation

Input : $\Sigma = \{\sigma_1, \ldots, \sigma_n\}$, sample chunk $\mathcal{D}_1$
Output : Selectivities s, result vectors R, similarity matrix MCC

for $i \leftarrow 1$ *to* n **do**
  $R[i] \leftarrow \text{ExecutePredicate}(\sigma_i, \mathcal{D}_1)$;
  $S[i] \leftarrow \frac{\text{CountTrue}(R[i])}{|\mathcal{D}_1|}$;
for $i \leftarrow 1$ *to* n **do**
  for $j \leftarrow i+1$ *to* n **do**
    $(TP, TN, FP, FN) \leftarrow \text{ConfusionCounts}(R[i], R[j])$;
    $\text{MCC}[i][j] \leftarrow \text{Matthews}(TP, TN, FP, FN)$;
return S, R, MCC

Algorithm 4: Candidate Path Generation with Dynamic Pruning

Input : s, MCC, Σ, μ_{thresh}, B, V
Output : Candidate paths $\mathcal{P}$

$p_{\text{sorted}} \leftarrow \text{SortAscendingBySelectivity}(\Sigma, S)$;
$\mathcal{P} \leftarrow \{\text{SeqPath}(p_{\text{sorted}})\}$;
$\mathcal{F} \leftarrow \emptyset$;
for $i \leftarrow 1$ *to* n **do**
  for $j \leftarrow i+1$ *to* n **do**
    if $\text{MCC}[i][j] > \mu_{thresh}$ **then**
      $\mathcal{F} \leftarrow \mathcal{F} \cup \{(\sigma_i, \sigma_j)\}$;
for *each* $(\sigma_a, \sigma_b) \in \mathcal{F}$ **do**
  $\sigma_a \oplus \sigma_b \leftarrow \text{Fuse}(\sigma_a, \sigma_b)$;
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{\text{SeqPath}(\text{ReplaceOnce}(p_{\text{sorted}}, \{\sigma_a, \sigma_b\} \rightarrow \sigma_a \oplus \sigma_b))\}$;
return $\mathcal{P}$

Algorithm 5: Path Exploration and Best-Path Selection

Input : Paths $\mathcal{P}$, sample chunk $\mathcal{D}_2$, threshold τ_{acc}
Output : Best path $p^\star$

Execute reference path to get ground truth G ;
for *each* $p \in \mathcal{P}$ **do**
  $(\text{lat}, \text{cost}, \text{sel}) \leftarrow \textsc{Run}(p, \mathcal{D}_2)$;
  $\text{acc} \leftarrow \textsc{Accuracy}(\text{sel}, G)$;
$\mathcal{F} \leftarrow \textsc{ParetoFrontier}(\mathcal{P}, \text{lat}, \text{cost})$;
$p^\star \leftarrow \arg\min_{p \in \mathcal{F}} \mathcal{M}[p].\text{lat}$;
return $p^\star$

C Benchmark Queries

Table 9: Profile of Query Set

Category	Subcategory	Query ID
Single Filter/Proj.	Single Operators	Q1–Q5
Consecutive Filters	Filter → Filter	Q6–Q10
Proj.-First	Proj. → Filter	Q11–Q13
	Proj. → Proj.	Q14–Q15
Filter-First	Filter → Proj.	Q16–Q18
	Filter → Aggregate	Q19–Q20

```
-- Q1: Sentiment analysis
SELECT TweetID AS id, sentiment, text FROM twitter LIMIT 32768
WHERE s'The sentiment label rules are:
- sentiment > 0 -> Positive
- sentiment = 0 -> Neutral
- sentiment < 0 -> Negative
Task: Judge whether the {sentiment} is consistent with the {text}
↪  content.');
```

Figure 11: Q1: Sentiment analysis

```
-- Q2: classify product into one of three categories
SELECT s'Classify the given {"Product Name"} into exactly one of
↪  these categories:
[Furniture, Office Supplies, Technology].
Output only the category name, with no explanations, no quotes,
↪  and no extra words.
Example:
Input: Hon Metal Bookcases, Black Output: Furniture
Input: AT&T CL2909 Output: Technology
Input: Belkin F9S820V06 8 Outlet Surge Output: Office Supplies'
↪  AS predicted_category FROM product;
```

Figure 12: Q2: Classify product into one of three categories

```
-- Q3: Determine whether crime's description is consistent with
↪  index code
SELECT iucr_no FROM IUCR
WHERE s'The index_code label rules are:
- index_code = I -> Indexed (severe, such as murder, rape, arson,
↪  and robbery)
- index_code = N -> Non-Indexed (less serve, such as vandalism,
↪  weapons violations, and peace disturbance)
Task: Determine whether the crime''s {primary_description} and
↪  {secondary_description} indicates this crime is consisted
↪  with {index_code}');
```

Figure 13: Q3: Determine whether crime's description is consistent with index code

```
-- Q4: Determine the risk degree according to violation
↪  description
SELECT s'Task: Classify the given violation {description} into
↪  exactly one of these categories:
- Low Risk
- Moderate Risk
- High Risk
Output only the risk category, with no explanations, no quotes,
↪  and no extra words.
Example:
Input: Inadequate and inaccessible handwashing facilities Output:
↪  Moderate Risk
Input: Unclean or degraded floors walls or ceilings Output: Low
↪  Risk
Input: Unclean or unsanitary food contact surfaces Output: High
↪  Risk' AS predicated_category, risk_category AS ground_truth
↪  FROM violations;
```

Figure 14: Q4: Determine the risk degree according to violation description

```sql
--Q5 Determine the fine degree according to violation description
SELECT inspection_id FROM (
    SELECT distinct inspection_id,inspector_comment, fine FROM
    ↪  violation
    WHERE inspector_comment IS NOT NULL
      AND fine IN [100, 250, 500]
    LIMIT 16384
)
WHERE s'The fine of more serious food safety problems will be
↪  higher.
The fine label rules are:
- fine = 100 -> Minor
- fine = 250 -> Serious
- fine = 500 -> Critical
Fine example:
1. {inspector_comment}: REPAIR THE DRAIN STOPPER FOR THE RIGHT
↪  COMPARTMENT OF THE 3 COMPARTMENT SINK SO THAT IT CAN BE USED
↪  TO HOLD WATER. {fine}: 100
2. {inspector_comment}: NO CHEMICAL TEST STRIPS ON PREMISES-MUST
↪  PROVIDE. SERIOUS VIOLATION 7-38-030. {fine}: 250
3. {inspector_comment}: FOUND WALK IN COOLER TEMPERATURE AT 45.3F.
↪  CRITICAL VIOLATION.7-38-005( A)  {fine}: 500
Task: Determine the {inspector_comment} is consistent with the
↪  {fine}. Output true or false only');
```

Figure 15: Q5: Determine the risk degree according to violation description

```sql
-- Q6: Find active cryptocurrencies that are still in circulation
↪  and have a big supply amount with a rising price.
SELECT description, coin.id AS coin_id FROM coins
WHERE category = 'coin' AND status = 'active' AND description is
↪  NOT NULL
AND s'The {description} shows that the cryptocurrency has a big
↪  supply amount.'
AND s'The {description} can prove that cryptocurrency is still in
↪  circulation.'
AND s'It can be inferred from the {description} that the price of
↪  this cryptocurrency is rising.';
```

Figure 16: Q6: Find active cryptocurrencies that are still in circulation but show weak community engagement and changing market volatility based on their descriptions

```sql
-- Q7: Find positive, valid user reviews of highly rated apps
↪  that are short or contain little information.
SELECT ur.Translated_review, ur.rowid AS ur_id FROM  user_reviews
↪  ur
JOIN playstore p ON ur.App = p.App
WHERE p.Rating >= 4.5
AND ur.Sentiment_Subjectivity IS NOT NULL AND
↪  ur.Sentiment_Polarity != 'nan'
AND CAST(ur.Sentiment_Polarity AS REAL) > 0.3
AND length(ur.Translated_Review) < 255
AND ur.Translated_review is NOT NULL
AND s'{ur.Translated_review} is a valid user review'
AND s'{ur.Translated_review} is positive'
AND s'{ur.Translated_Review} is very short or low-information';
```

Figure 17: Q7: Find positive, valid user reviews of highly rated apps that are short or contain little information

```sql
-- Q8: Find valid negative or neutral user reviews of free apps
↪  that report problem and show low subjectivity (objective
↪  tone).
SELECT ur.id AS user_review_rowid FROM user_reviews ur
JOIN playstore p ON ur.App = p.App
WHERE p.Type = 'Free'
AND ur.Sentiment_Subjectivity IS NOT NULL
AND ur.Translated_Review != 'nan'
AND length(ur.Translated_Review) < 255
AND CAST(ur.Sentiment_Subjectivity AS REAL) < 0.3
AND s'{Translated_review} is a valid user review (which is a
↪  complete sentence and has informative content)'
AND s'{Translated_review} is a negative or neutral user review'
AND s'{Translated_Review} reports someone has a problem with this
↪  app'
```

Figure 18: Q8: Find valid negative user reviews of free apps that report bugs or crashes and show low subjectivity (objective tone)

```sql
-- Q9: Find valid positive or neutral user review that shows this
↪  app is very creative and innovative.
SELECT id AS id
FROM (SELECT id, Translated_Review
      FROM user_reviews ur JOIN playstore p
      ON ur.App = p.App AND p.Type = 'Free'
      AND ur.Sentiment_Subjectivity IS NOT NULL
      AND ur.Translated_Review != 'nan'
      AND length(ur.Translated_Review) < 255
      AND CAST(ur.Sentiment_Subjectivity AS REAL) < 0.3)
WHERE s'{Translated_Review} is a valid user review (which is a
↪  complete sentence and has informative content)'
AND s'{Translated_Review} is a positive or neutral user review'
AND s'{Translated_Review} shows that this app is very creative
↪  and innovative';
```

Figure 19: Q9: Find valid positive or neutral user review that shows this app is very creative and innovative

```sql
-- Q10: Find released movies with short overviews that describe stories set in the past, centered on a specific profession or role, and based on historical events or folk tales.
SELECT m.movie_id AS movie_id FROM movie m
SEMI JOIN
  (SELECT movie_id
   FROM movie_genres mg JOIN genre g ON mg.genre_id = g.genre_id
  ) mg ON m.movie_id = mg.movie_id
  AND length(overview) < 255 AND m.overview IS NOT NULL AND m.movie_status = 'Released'
WHERE s'The {overview} shows that the movie is set in the past'
AND s'The {overview} shows that the movie focuses on a central role'
AND s'The {overview} shows that the movie is based on historical events or a folk story';
```

Figure 20: Q10: Find released movies with short overviews that describe stories set in the past, centered on a specific profession or role, and based on historical events or folk tales.

```sql
-- Q11: Find cryptocurrencies that were inactive or untracked on Jan 9, 2019, had missing price data, and whose description suggests they were still under development or operating on a testnet.
SELECT * FROM
(SELECT s'According to {c.description},introduce the situation of cryptocurrency circulation' AS circulation
    coin.id AS coin_id
    FROM coins c JOIN historical h ON h.coin_id = c.id
    AND h.date = '2019-01-09'
    AND c.status IN ('untracked','inactive')
    AND c.description IS NOT NULL)
WHERE s'{circulation} indicates cryptocurrency is under development or in testnet.';
```

Figure 21: Q11: Find cryptocurrencies that were inactive or untracked on Jan 9, 2019, had missing price data, and whose description suggests they were still under development or operating on a testnet

```sql
-- Q12: 1. Find coins whose he summarized technological features indicate the cryptocurrency has its own blockchain or consensus mechanism and extract information.
SELECT * FROM
(SELECT s'From the description {description}, extract information about the underlying technology of cryptocurrency, focusing on whether it mentions a proprietary blockchain, a native consensus mechanism, or an independent network.Provide a concise summary of these technological characteristics.' AS tech_features,
    coins.id AS coin_id
    FROM coins c JOIN historical h ON h.coin_id = c.id
    WHERE c.platform_id IS NOT NULL AND c.description IS NOT NULL
    AND h.circulating_supply IS NOT NULL
    AND h.total_supply IS NOT NULL
    AND h.date = '2019-01-10'
    )
WHERE s'Based on the technological characteristics {tech_features}, determine whether cryptocurrency has its own independent blockchain or consensus mechanism';
```

Figure 22: Q12: Find coins whose summarized technological features indicate the cryptocurrency has its own blockchain or consensus mechanism and extract information about its underlying technology and extract information

```sql
-- Q13: Find profitable movies whose background indicates the story is set in modern times rather than historical periods with limited genres.
SELECT movie_id FROM
(SELECT s'Summarize the geographic and temporal background of the movie according to {overview}.' AS background, m.movie_id AS movie_id
    FROM movie m
    SEMI JOIN (
      SELECT mg.movie_id
      FROM movie_genres mg JOIN genre g ON mg.genre_id = g.genre_id
      WHERE g.genre_name IN ('Thriller','Comedy','Western')
    ) mg ON m.movie_id = mg.movie_id
    WHERE revenue > budget
      AND length(overview) < 255
 )
WHERE s'{background} indicates the story is set in modern times rather than historical periods.';
```

Figure 23: Q13: Find profitable movies whose background indicates the story is set in modern times rather than historical periods with limited genres

```sql
-- Q14 Summarize the main plot of English released popular movies and classify plots into genres
SELECT movie_id, s'Classify {plot} into one of the following categories:
[Sci-Fi, Crime, Thriller, Romance, Drama, Horror, Comedy, War, Fantasy, Family, Animation, Biography, Other]
Output only the category that best fits the movie, without any other text or reasoning content.' AS category FROM
(SELECT s'Summarize the main plot of {overview}' AS plot, movie_id
    FROM movie m
    SEMI JOIN (
        SELECT ml.movie_id
        FROM movie_languages ml
        JOIN "language" l ON ml.language_id = l.language_id AND l.language_name = 'English'
    ) ml ON m.movie_id = ml.movie_id AND movie_status = 'Released' AND popularity > 10 AND length(overview) < 255 AND length(overview) > 0
);
```

Figure 24: Q14: Summarize the main plot of English released popular movies and classify plots into genres

```sql
-- Q15: 1. Extract all conflicts from movies high-rated movies between 2000 and 2015. 2. Classify conflict into these categories
SELECT movie_id,
s'This is a multi-label classification task. Given the {conflict}, identify what the main character is in conflict with, and classify the it in the format "character vs X", where X is one or more of: [Character, Self, Society, Nature, Technology, Machine, Supernatural, Fate, Other].
Output only X, as a comma-separated list (e.g., Self, Society).
Do not include explanations or reasoning.' AS categories
FROM (SELECT movie_id, s'Extract all conflicts from movie {overview} as one sentence.' AS conflict
    FROM movie
    SEMI JOIN
        (SELECT pc.movie_id
         FROM production_country pc JOIN country c
         ON pc.country_id = c.country_id AND c.country_iso_code = 'US')
    pc ON movie.movie_id = pc.movie_id AND (movie.release_date BETWEEN '2000-01-01' AND '2015-12-31') AND movie.vote_average > 6.0 AND length(overview) < 255;
```

Figure 25: Q15: Extract all conflicts from high-rated movies between 2000 and 2015 and classify conflicts into categories

```sql
-- Q16: 1. Filter neutral tweets which are posted by females and related to recruitment 2. Extract concrete job titles and technical skills.
SELECT s'From the job posting tweet {t.text}, extract the specific job titles and required skills mentioned. Return them as a comma-separated list (e.g., Software Engineer, Python, AWS, Machine Learning) Focus on concrete job titles and technical skills' AS info,t.TweetID AS twitter_id
FROM twitter t JOIN user u
On t.UserID = u.UserID
WHERE user.Gender = 'Female'
AND t.Sentiment = 0.0
AND t.TweetID IS NOT NULL
AND Klout > 50
AND s'Filter tweets "{t.text}" that are job postings, recruitment announcements, hiring notices, or career opportunities related to AWS or cloud computing (looking for phrases like hiring, job opening, career opportunity, we are looking for, join our team, etc.)';
```

Figure 26: Q16: Filter neutral tweets which are posted by females and related to recruitment and extract concrete job titles and technical skills.

```sql
-- Q17: 1. Filter negative tweets with high influence that are related to potential risks of AWS services 2. Identify the main type of security or privacy concern
SELECT s'Based on the tweet content :"{text}", identify the main type of security or privacy concern being expressed.
Classify into categories like data breach, unauthorized access, encryption weakness, compliance violation, identity theft, service vulnerability, configuration error, monitoring concerns, or other specific security/privacy concern types.' AS type,
twitter.TweetID AS twitter_id
FROM twitter
WHERE Sentiment < -2.5 AND Klout > 20
AND t.TweetID IS NOT NULL
AND s'Filter tweets "{text}" that discuss security concerns, privacy issues, data protection, cybersecurity threats, authentication problems, or confidentiality matters related to AWS services';
```

Figure 27: Q17:Filter negative tweets with high influence that are related to potential risks of AWS services and identify the main type of security or privacy concern

```
-- Q18: 1. Filter positive tweets related to AWS products 2.
↪  Extract discussed topics or subjects
SELECT s'From this tweet "{t.text}", extract the main topic or
↪  subject being discussed about programming/development. Return
↪  a concise topic label like performance optimization, learning
↪  resources, job opportunities, technical issues, new features,
↪  etc.'
twitter.TweetID AS twitter_id
FROM twitter t JOIN location l
On t.LocationID = l.LocationID
WHERE l.Country IN ('United States', 'Canada', 'United Kingdom')
AND Sentiment >= 3.0 AND Reach>1000
AND t.TweetID IS NOT NULL
s'Filter tweets "{t.text}" that mention amazon web services or
↪  cloud services, including specific AWS products or general
↪  cloud computing issues';
```

Figure 28: Q18: Filter tweets related to AWS products and extract discussed topics or subjects

```
-- Q19: 1. Find papesr of Database journals and related to
↪  database systems, data management, or query processing'
-- 2.Aggregate these papers' keywords to find main research themes
SELECT sem_agg('Summarize the main research theme reflected by
↪  the following keywords: {p.Keyword}. Respond with a concise,
↪  high-level scientific topic',Keyword) FROM Paper p
JOIN Journal j
ON j.Id= P.JournalId
WHERE p.Year BetWEEN 2000 AND 2011
AND j.ShortName IN ('TODS','VLDB','TKDE','JDM','DKE')
AND s'Based on the keywords "{p.Keyword}" , determine if this
↪  paper is related to database systems, data management, or
↪  query processing';
```

Figure 29: Q19: Find papers of Database journals and related to database systems, data management, or query processing and aggregate these papers' keywords to find main research themes

```
-- Q20: 1. Find papers whoses authors is in institute, laboratory
↪  or research center
-- 2.Filter papers related to interdisciplinary methods
-- 3.Identify and summarize the major interdisciplinary research
↪  trends for each conference
SELECT ConferenceId, sem_agg('Identify and summarize the major
↪  interdisciplinary research trends from the following keywords:
↪  {p.Keyword}. Return only one sentence.',Keyword) AS trends
FROM paper p
JOIN Conference c ON p.ConferenceId = c.Id
SEMI JOIN (
SELECT DISTINCT pa.PaperId
  FROM PaperAuthor pa
  WHERE pa.AuthorId IS NOT NULL
    AND (LOWER(COALESCE(pa.Affiliation,'')) LIKE '%institute%'
    OR LOWER(pa.Affiliation) LIKE '%laboratory%'
    OR LOWER(pa.Affiliation) LIKE '%research center%')
) filtered_papers
ON p.Id = filtered_papers.PaperId
AND p.Keyword IS NOT NULL
AND length(p.Keyword) > 0
WHERE s'Determine whether the paper with keyword as {Keyword} is
↪  related to interdisciplinary methods.'
GROUP BY ConferenceId
HAVING COUNT(*) >= 3 AND trends IS NOT NULL
```

Figure 30: Q20: Find papers whose authors are in institute, laboratory or research center and identify interdisciplinary research trends

```
-- Note: Introduces redundant expressions - repeated field
↪  references ({primary_description}, {secondary_description},
↪  {index_code} each appear twice), redundant modifiers ("after
↪  reviewing...again for consistency"), and complex sentence
↪  structures.
SELECT COUNT(*) FROM chicago_crime.IUCR
WHERE s'The index_code label rules are:\n\
- index_code = I -> Indexed (severe, such as murder, rape, arson,
↪  and robbery)\n\
- index_code = N -> Non-Indexed (less serve, such as vandalism,
↪  weapons violations, and peace disturbance)\n\
Based on these rules, the crime with primary description
↪  {primary_description} and secondary description
↪  {secondary_description} should be checked against index code
↪  {index_code}, and after reviewing the primary description
↪  {primary_description} again for consistency, it should be
↪  determined whether this crime is consistent with the assigned
↪  index code {index_code} according to the established
↪  classification criteria.';
```

Figure 31: Q3 before Expression Reduction

```
-- Note: Deliberately uses incorrect column name references -
↪  replaces correct {primary_description} and
↪  {secondary_description} with non-existent {primary_desc} and
↪  {second_descr}.
SELECT COUNT(*) FROM chicago_crime.IUCR
WHERE s'The index_code label rules are:\n\
- index_code = I -> Indexed (severe, such as murder, rape, arson,
↪  and robbery)\n\
- index_code = N -> Non-Indexed (less serve, such as vandalism,
↪  weapons violations, and peace disturbance)\n\
Task: Determine whether the crime''s {primary_desc} and
↪  {second_descr} indicates this crime is consisted with
↪  {index_code}';
```

Figure 32: Q3 before Meta-entity Resolution

```
-- Note: Introduces repeated field references ({description}
↪  appears three times) and redundant sentence structures
↪  (repetitive "and the {description} indicates that..."
↪  patterns).
SELECT coins.id AS coin_id FROM coins
WHERE category = 'coin' AND status = 'active' AND description is
↪  NOT NULL
AND s'The {description} shows that this cryptocurrency has a big
↪  supply amount and the {description} indicates that this
↪  cryptocurrency is still in circulation and the {description}
↪  can be used to infer that the price of this cryptocurrency is
↪  rising.';
```

Figure 33: Q6 before Expression Reduction

```
-- Note: Mixes correct and incorrect column names - introduces
↪  non-existent {text} field reference alongside {description}
SELECT coins.id AS coin_id FROM coins
WHERE category = 'coin' AND status = 'active' AND description is
↪  NOT NULL
AND s'The {description} shows that the cryptocurrency has a big
↪  supply amount and the {text} indicates that this
↪  cryptocurrency is still in circulation and the {description}
↪  can be used to infer that the price of this cryptocurrency is
↪  rising.';
```

Figure 34: Q6 before Meta-entity Resolution

```
-- Note: Adds redundant modifiers ("which describes the film
↪  narrative"), repeated field references ({overview} appears
↪  twice), and complex expressions ("after reviewing...for
↪  temporal clues").
SELECT movie_id FROM
    (SELECT s'Based on the movie overview {overview} provided,
    ↪  which describes the film narrative, summarize the
    ↪  geographic and temporal background of the movie,
    ↪  including specific locations and time periods mentioned
    ↪  in this overview {overview}.' AS background, movie_id
        FROM movie
        SEMI JOIN (
            SELECT mg.movie_id
            FROM movie_genres mg JOIN genre g ON mg.genre_id =
            ↪  g.genre_id
            WHERE g.genre_name IN ('Thriller','Comedy','Western')
        ) mg ON m.movie_id = mg.movie_id
        WHERE revenue > budget
        AND length(overview) < 255
    )
    WHERE s'The background summary {background} indicates the
    ↪  story is set in modern times rather than historical
    ↪  periods, and after reviewing the background {background}
    ↪  for temporal clues, this setting classification is
    ↪  confirmed.';
```

Figure 35: Q13 before Expression Reduction

```
-- Note: Uses misspelled column name {overvie} and incomplete
↪  alias reference {back} (should be {background}).
SELECT movie_id FROM
(SELECT s'Summarize the geographic and temporal background of the
↪  movie according to {overvie}.' AS background, movie_id
    FROM movie
    SEMI JOIN (
        SELECT mg.movie_id
        FROM movie_genres mg JOIN genre g ON mg.genre_id =
        ↪  g.genre_id
        WHERE g.genre_name IN ('Thriller','Comedy','Western')
    ) mg ON m.movie_id = mg.movie_id
    WHERE revenue > budget
    AND length(overview) < 255
)
WHERE s'{back} indicates the story is set in modern times rather
↪  than historical periods.';
```

Figure 36: Q13 before Meta-entity Resolution

```
-- Note: Uses incorrect column names {synopsis} and {plt} (should
↪  be {overview} and {plot}).
SELECT plot,s'Classify {plt} into one of: Sci-Fi, Crime, Thriller,
↪  Romance, Drama, Horror, Comedy, War, Fantasy, Family,
↪  Animation, Biography, Other' AS category
FROM
(SELECT s'Summarize the main plot of {synopsis}' AS plot
    FROM movie m
    SEMI JOIN (
        SELECT ml.movie_id
        FROM movie_languages ml
        JOIN "language" l ON ml.language_id = l.language_id AND
        ↪  l.language_name = 'English'
    ) ml ON m.movie_id = ml.movie_id AND movie_status =
    ↪  'Released' AND popularity > 10 AND length(overview) < 255
    ↪  AND length(overview) > 0
);
```

Figure 39: Q14 before Meta-entity Resolution

```
-- Note: Replaces originally SQL-expressible numerical comparison
↪  predicate popularity > 10 with natural language description
↪  'the {popularity} is greater than 10'.
SELECT plot,s'Classify {plot} into one of: Sci-Fi, Crime,
↪  Thriller, Romance, Drama, Horror, Comedy, War, Fantasy,
↪  Family, Animation, Biography, Other' AS category
FROM
(SELECT s'Summarize the main plot of {overview}' AS plot FROM
↪  movie
    FROM movie m
    SEMI JOIN (
        SELECT ml.movie_id
        FROM movie_languages ml
        JOIN "language" l ON ml.language_id = l.language_id AND
        ↪  l.language_name = 'English'
    ) ml ON m.movie_id = ml.movie_id AND movie_status =
    ↪  'Released' AND popularity > 10 AND length(overview) < 255
    ↪  AND length(overview) > 0
);
```

Figure 40: Q14 before Predicate Deduction

```
-- Note: Adds redundant explanatory text ("begin by
↪  examining..."), repeated field references ({text} appears
↪  three times), and complex classification descriptions.
SELECT TweetID AS twitter_id, s'begin by examining the tweet
↪  content: "{text}". Based on a comprehensive review of this
↪  tweet content {text}, identify and determine the main type of
↪  security or privacy concern being expressed or referenced in
↪  the message. Then, proceed to classify and categorize it into
↪  one of the specified categories such as: data breach,
↪  unauthorized access, encryption weakness, compliance
↪  violation, identity theft, service vulnerability,
↪  configuration error, monitoring concerns, or other.' AS type
↪  FROM twitter
WHERE Sentiment < -2.5 AND Klout > 20
AND TweetID IS NOT NULL
AND s'The tweet content {text} discusses security concerns,
↪  privacy issues, data protection, cybersecurity threats,
↪  authentication problems, or confidentiality matters, and
↪  after analyzing the content {text} for AWS relevance, it is
↪  confirmed that these discussions are related to AWS services
↪  and their security aspects.';
```

Figure 41: Q17 before Expression Reduction

```
-- Note: Uses inconsistent column name references - main query
↪  uses \verb|{tweet_content}|, filter condition uses
↪  \verb|{txt}|, while actual column name is \verb|text|.
SELECT TweetID AS twitter_id, s'Based on the tweet content
↪  :"{tweet_content}", identify the main type of security or
↪  privacy concern being expressed.
Classify it into one of the categories like [data breach,
↪  unauthorized access, encryption weakness, compliance
↪  violation, identity theft, service vulnerability,
↪  configuration error, monitoring concerns, other].
Output only the category that best fits the tweet, without any
↪  other text or reasoning.'
AS type FROM twitter
WHERE Sentiment < -2.5 AND Klout > 20
AND TweetID IS NOT NULL
AND s'Tweets with content as {txt} discuss security concerns,
↪  privacy issues, data protection, cybersecurity threats,
↪  authentication problems, or confidentiality matters related
↪  to AWS services';
```

Figure 42: Q17 before Meta-entity Resolution